

AML Admin Portal — Functional & Non-Functional Requirements Specification

Executive Summary

This document specifies the functional and non-functional requirements for an AML (Anti-Money Laundering) admin portal comprising four integrated modules: a KPI Monitoring and Alerting Dashboard, a Case Management system, an Interactive Network Visualization layer for fund flow analysis, and an ETL pipeline serving a PostgreSQL transaction database and Apache AGE graph exploration layer. The specification targets the established stack of Next.js + React + TypeScript on the frontend and Python + FastAPI + PostgreSQL + Apache AGE + Temporal on the backend.[1][2][3]

The system must satisfy global regulatory frameworks including the Bank Secrecy Act (BSA), EU AML Directives (AMLD5 and AMLD6), FATF recommendations, and GDPR, all of which mandate transaction monitoring, suspicious activity reporting, audit documentation, and risk-based controls.[4][5][3]

Module 1: KPI Monitoring and Alerting Dashboard

Functional Requirements

FR-D01 — KPI Metrics Display

The dashboard must display the following AML operational KPIs in real time, updated at intervals not exceeding 60 seconds for live alert queues:[6][7][8][9]

- Alert volume (total, by rule, by risk tier, by channel)

- Alert-to-case conversion rate
- False-positive rate and trend line
- SAR/STR filing volume and status
- Average case resolution time (time-to-close)
- Queue backlog count and aging distribution
- SLA compliance percentage per investigator and team
- Customer risk tier breakdown (low, medium, high, PEP, sanctions)
- KYC completion rate
- Cost-per-case (computed from case count and team hour inputs)

FR-D02 — Traffic-Light Indicators

Each KPI card must display a three-level traffic-light indicator (green, amber, red) against configurable thresholds, providing at-a-glance health status for the AML/CFT compliance officer and management. Threshold values must be configurable per role without requiring a code deployment.[9][6]

FR-D03 — Alert Volume Charts

The dashboard must render time-series charts for alert volume, SAR filing volume, and false-positive rate across configurable rolling windows (24 h, 7 d, 30 d, 90 d). Charts must support zoom, pan, and data-point inspection, and must use ECharts or a comparable library capable of handling large time-series datasets.[8][10]

FR-D04 — Tiered Alert Notification

The system must deliver automated notifications using a tiered strategy to prevent alert fatigue:[9]

- **Critical** (high-risk, threshold-amount breach): immediate push notification and email to assigned compliance officer
- **High** (SLA risk, escalation pending): in-app notification and email digest every 2 hours
- **Medium** (routine monitoring pattern): aggregated daily digest
- **Low / Informational**: visible in-portal only, no push

Each tier's threshold conditions must be configurable by compliance administrators.

FR-D05 — Analyst Operational View

The dashboard must provide a dedicated operational view for transaction monitoring analysts showing alert volume, backlog, alerts resolved, alerts escalated, and a throughput trend, updated in real time. Managers must see aggregated team-level views.[10]

FR-D06 — Rule and Scenario Performance

The dashboard must display per-rule performance metrics including: hit rate, false-positive rate, number of cases generated, and last modification timestamp. Rules with a false-positive rate above a configurable threshold must be highlighted automatically.[7]

FR-D07 — Exportable Compliance Reports

Compliance officers must be able to export dashboard snapshots and metric summaries as PDF and CSV for board reporting, regulator inspection, and internal audit.[11][9]

Frontend Requirements (Module 1)

Requirement	Detail
Technology	Next.js App Router page, TanStack Query for polling/cache, ECharts for time-series
Refresh strategy	TanStack Query refetchInterval for live KPI cards; WebSocket or SSE for critical alert counters
Layout	KPI cards row → trend charts → alert queue summary table, following top-KPI-then-detail hierarchy
Responsive	Full-width at 1280px+; KPI cards stack vertically at ≤768px
Skeleton states	All KPI cards and charts must render shimmer skeletons during load
Accessibility	WCAG AA contrast, keyboard navigable, screen-reader labels on all chart elements

Backend Requirements (Module 1)

Endpoint	Method	Description
/dashboard/kpis	GET	Returns all current KPI values with computed traffic-light status
/dashboard/alerts/timeseries	GET	Returns alert volume time-series for a given window and granularity
/dashboard/rules/performance	GET	Returns per-rule hit rate, FP rate, case conversion
/dashboard/alerts/notifications	GET (SSE or WS)	Streams critical alert events to connected clients
/dashboard/export	POST	Generates PDF or CSV report snapshot

Module 2: Case Management

Functional Requirements

FR-C01 — Case Creation

Cases must be createable by two triggers:[12][2][13]

1. **Automatic:** an alert fired by the transaction monitoring rules engine, with case metadata pre-populated from the alert payload (entity, rule ID, alert severity, triggering transaction IDs)
2. **Manual:** an investigator or supervisor creates a case from the alert queue, a graph investigation, or a scheduled review

FR-C02 — Case Lifecycle Workflow

Each case must follow a defined lifecycle with configurable stage transitions:[2][3]

- NEW → TRIAGED → UNDER_INVESTIGATION → ESCALATED → CLOSED (cleared or SAR filed) or REOPENED
- Workflow transitions must enforce role-based gate conditions (e.g., only a supervisor can approve escalation)

- Stage durations must be tracked for SLA monitoring

FR-C03 — Role-Based Access Control (RBAC)

The system must implement a role matrix with at minimum the following roles:[3][14]

- **Analyst:** create, view, edit assigned cases; add notes; request escalation
- **Supervisor:** view all team cases; approve/reject escalations; override dispositions
- **QA Reviewer:** read-only access to closed cases for quality control auditing
- **Compliance Officer (MLRO):** view all cases; approve SAR filing; access board-level reporting
- **Compliance Admin:** configure rules, thresholds, workflow stages, and user assignments
- **System Admin:** manage users, roles, integrations, and audit log access

No role may exceed its defined permissions; privilege escalation must require MLRO or admin approval.[3]

FR-C04 — Case Workspace

Each case must have a workspace containing:[13][12][2]

- Case summary (entity, risk score, alerts linked, assigned investigator, status, age)
- Transaction evidence viewer (linked transactions with amounts, timestamps, counterparties)
- Graph investigation panel (embedded Cytoscape.js fund-flow view for case entities)
- Timeline of all case actions in chronological order
- Notes and tagging with multi-investigator collaboration support
- File attachment for supporting evidence (KYC documents, screenshots, external reports)
- SAR/STR preparation form with regulatory template fields

FR-C05 — SAR/STR Generation

The system must support generation of SAR (Suspicious Activity Reports) and STR (Suspicious Transaction Reports) pre-populated with case data, supporting jurisdiction-specific templates. Completed reports must be exportable in the format required by the relevant financial intelligence unit (FIU). Filing status and confirmation must be recorded in the case audit trail.[4][13]

FR-C06 — Audit Trail and Chain of Custody

Every case action must be persisted as an immutable, append-only audit event including:[11][15][3]

- Action type (create, edit, note, status change, file attach, SAR file, escalate, close)
- Actor (user ID, role, IP address)
- Timestamp (UTC, millisecond precision)
- Before/after state for field changes
- Rule version and model version in effect at time of alert
- Approval workflow log and MLRO sign-off

Audit records must be stored in a tamper-evident log and must meet FATE, AMLD5/6, and EBA documentation requirements.[15][3]

FR-C07 — SLA and KPI Tracking

The system must track per-case SLA targets and surface breaches in both the case workspace and the dashboard:[16][3]

Priority	Response Target	Resolution Target
Critical (P1)	15 minutes	4 hours (24/7)
High (P2)	30 minutes	8 hours (24/7)
Medium (P3)	1 hour	1 business day
Low (P4)	2 hours	2 business days

FR-C08 — Case Linking and Entity Resolution

Cases involving overlapping entities (shared account, device, customer, jurisdiction) must be surfaced as related cases and linkable into a case cluster for joint investigation. Entity resolution must allow analysts to merge or link entities across cases when duplicates are identified.[3]

FR-C09 — Integration Points

The case management module must integrate via API with:[12][2]

- Transaction monitoring rules engine (alert ingestion)
- KYC/CDD provider (customer risk profile enrichment)
- Sanctions and PEP screening API (entity screening at case open and on demand)
- Graph investigation service (network subgraph for case entities)
- Regulatory reporting system (SAR/STR export and submission confirmation)

Frontend Requirements (Module 2)

Require ment	Detail
Technolo gy	Next.js, TanStack Table for case queues, React Hook Form + Zod for case forms and SAR templates
Case queue	Faceted filter sidebar (risk, status, rule, assignee, date); column-sortable TanStack Table; bulk disposition actions
Case workspac e	Split-pane layout: left summary + evidence; right notes + timeline; full-screen graph investigation modal
Notificati ons	In-app toast for SLA breach, escalation, and assignment; SSE feed from backend
Optimisti c updates	Case status and note additions must optimistically update before server confirmation

Backend Requirements (Module 2)

Endpoint	Method	Description
/cases	GET / POST	List cases with filters; create manual case
/cases/{id}	GET / PATCH / DELETE	Retrieve, update, or soft-delete a case
/cases/{id}/transitions	POST	Execute workflow state transition with role validation
/cases/{id}/notes	GET / POST	List or add case notes
/cases/{id}/attachments	GET / POST	List or upload case attachments
/cases/{id}/audit	GET	Retrieve immutable audit log for the case
/cases/{id}/sar	POST	Generate or submit SAR/STR document
/cases/{id}/graph	GET	Return Cytoscape-ready subgraph for case entities
/alerts	GET	List alerts with filtering and pagination
/alerts/{id}/case	POST	Convert alert to case

Module 3: Interactive Network Visualization (Fund Flow Analysis)

Functional Requirements

FR-G01 — Entity and Relationship Model

The graph visualization must represent AML-relevant entities and relationships:[17][18][19]

Node types: Account, Customer, Merchant, Device, Wallet, Transaction, Case

Edge types: SENT, RECEIVED, OWNS, CONTROLS, SHARES_DEVICE, SHARES_IP, LINKED_TO_CASE, INTERMEDIARY_FOR

Each node must display: entity ID, type label, risk score (color-coded), jurisdiction flag, KYC status indicator, and case count badge. Each edge must display: amount (or count for aggregated edges), direction, timestamp range, and suspicious flag.

FR-G02 — Interactive Traversal

The investigation canvas must support:[20][18][19]

- Seed-node selection by entity search or from a case workspace
- Configurable hop expansion (1–4 hops, defaulting to 2) with expansion triggered by double-click or context menu
- Shortest path trace between any two nodes
- Circular flow detection highlighting (fund paths that return to the origin node)
- Fan-in / fan-out view (entities sending to or receiving from many counterparties)
- Collapse benign branches into a summary node
- Filter by time window, amount threshold, currency, jurisdiction, edge type, and risk score

FR-G03 — AML Pattern Highlighting

The system must programmatically highlight known typologies on the graph:[18][21][20]

- **Structuring/Smurfing:** one node sending many sub-threshold transactions to multiple destinations (fan-out)
- **Layering:** long chain of intermediate accounts with rapid successive transfers
- **Integration:** funds converging from many sources into a single endpoint (fan-in)

- **Round-tripping:** circular paths where funds return to the originating account or beneficial owner
- **Hub/bridge account:** a node with unusually high betweenness centrality connecting otherwise disconnected clusters

Detected patterns must be marked with distinct visual indicators and listed in the investigation sidebar.

FR-G04 — Graph Layout and Navigation

The canvas must support at minimum two layout modes:[22][23]

- **Force-directed** (fCoSE) for relationship exploration
- **Hierarchical / top-down** for fund-flow chain tracing

Navigation must support zoom (mouse scroll and pinch), pan (click-drag), fit-to-view, and full-screen mode. Large graphs must use progressive rendering and batched layout to maintain interactive response times (see Non-Functional Requirements).

FR-G05 — Node and Edge Inspection

Clicking a node or edge must open a details panel without leaving the canvas, showing:[20]

- Full entity attributes and risk history
- Linked alerts and cases
- Transaction list for the selected relationship (sorted by amount and date)
- Option to "open in case" (create or link to a case from the node)

FR-G06 — Graph Snapshot and Evidence Export

Analysts must be able to export the current visible graph as a PNG image or JSON elements file for inclusion in case evidence, SAR submissions, or regulatory reports.[11][20]

Frontend Requirements (Module 3)

Requirement	Detail
Technology	Cytoscape.js for graph rendering; React wrapper for event integration
Performance	Batch element addition via cy.batch(); cap default rendered elements at 1,000 (nodes + edges); use progressive expansion beyond that limit[23][24]
Layouts	fCoSE for force-directed; dagre for hierarchical; layout computed server-side for large subgraphs to shift CPU cost off the browser[23]
Edge aggregation	Multiple transactions between the same pair should render as one weighted edge with count and total amount, expandable on demand
Risk color coding	Node color encodes risk score: green (low), amber (medium), red (high), black (sanctioned/PEP)
Undo/redo	Canvas state changes (expand, collapse, filter) support undo/redo via an in-memory state stack

Backend Requirements (Module 3)

Endpoint	Method	Description
/graph/neighborhood/{entity_id}	GET	Returns 1–N hop subgraph centered on entity; ?depth=2 (max 3)
/graph/path	GET	Shortest path between from and to entity IDs within maxHops
/graph/case/{case_id}	GET	Full subgraph for all entities linked to a case
/graph/expand	POST	Expand from a set of seed nodes with time/amount/type filters
/graph/patterns/{entity_id}	GET	Returns detected AML patterns (structuring, layering, circular) for entity neighborhood

All graph endpoints return Cytoscape-compatible elements JSON with nodes and edges arrays, plus a meta object with depth, truncation flag, pattern detections, and aggregate totals.

Apache AGE Cypher queries power path, neighborhood, and pattern endpoints. Regular PostgreSQL SQL handles aggregation, scoring, and filter pre-processing before graph traversal.[25][18][26]

Module 4: ETL Pipeline (PostgreSQL Transaction DB + Apache AGE Graph Layer)

Functional Requirements

FR-E01 — Data Source Ingestion

The ETL pipeline must extract data from the following source types: [27][28][29]

- **Real-time streaming:** Apache Kafka topics (core banking transaction events, payment authorizations, alert triggers)
- **SWIFT and ISO 20022:** message parsing for MT/MX payment messages

- **REST API polling:** KYC provider enrichment, sanctions/PEP list updates, adverse media feeds
- **Batch file:** CSV, JSON, and XML files from core banking, ledger export, and historical migration
- **CDC (Change Data Capture):** incremental updates from source relational databases via Debezium or equivalent

FR-E02 — Transformation and Enrichment

The transformation stage must:[28][30][27]

- Normalize entity identifiers across sources (account number, IBAN, wallet address, national ID) into stable canonical entity IDs
- Resolve duplicate and aliased entities using configurable match rules (exact, fuzzy, deterministic)
- Enrich transactions with KYC/CDD risk scores, PEP status, sanctions flags, and jurisdiction codes
- Parse and standardize date/time fields to UTC ISO 8601
- Validate required fields against the AML data model schema (Party, AccountPartyLink, Transaction, Risk tables)[31][32]
- Apply data quality checks with rejection and quarantine for records failing validation
- Compute derived fields: transaction velocity indicators, peer-group deviation scores, running-balance snapshots

FR-E03 — Load to PostgreSQL (Transaction Layer)

The load stage must write to PostgreSQL with the following guarantees:[1][33][34]

- Idempotent upserts (no duplicate records on retry or redelivery)
- Transactional writes per micro-batch with rollback on partial failure
- Table partitioning by date on the Transaction table for query performance
- Separate schemas for: core_banking, aml_staging, aml_clean, risk, case_management, audit
- Retention policy hooks for regulatory data holds and GDPR erasure requests

FR-E04 — Load to Apache AGE (Graph Exploration Layer)

After relational load, a graph projection job must mirror investigation-relevant entities and relationships into the aml_graph Apache AGE graph:[35][25][36]

- **Upsert vertices** for: Account, Customer, Merchant, Device, Wallet, Case
- **Upsert edges** for: SENT, RECEIVED, OWNS, CONTROLS, SHARES_DEVICE, LINKED_TO_CASE — with properties including amount, count, first_seen, last_seen, currency, risk_flag, transaction_ids
- Incremental projection: only changed or new entities and relationships are projected per pipeline run, not full graph rebuild
- Graph projection must run within the same PostgreSQL transaction as the relational load where atomicity is required, leveraging AGE's PostgreSQL-native transactional model[26]

FR-E05 — Pipeline Orchestration

Pipeline orchestration must use Apache Airflow for DAG-based scheduling and dependency management, given its maturity for enterprise batch ETL and extensive community operator ecosystem. DAGs must be defined for:[37][38][39]

- Real-time/micro-batch transaction ingestion (trigger: Kafka consumer)
- Batch file processing (trigger: cron schedule or file arrival)
- Sanctions/PEP list refresh (trigger: daily cron)
- Graph projection job (trigger: completion of relational load DAG)
- Data quality report generation (trigger: post-load)
- Historical back-fill and reprocessing (trigger: manual or rule-change event)

FR-E06 — Data Quality and Observability

The pipeline must:[30][28]

- Report data quality metrics per pipeline run: record count, reject count, duplicate count, enrichment hit rate

- Target ≥99% data quality accuracy per run, consistent with scalable ETL pipeline benchmarks in the literature[28]
- Alert the data engineering team when quality falls below the 99% threshold or when a pipeline stage fails
- Maintain pipeline lineage records mapping each output record to its source file, topic, and transformation version
- Support on-demand reprocessing of specific time windows or message types via a Rerun Engine[27]

FR-E07 — Schema and Graph Model

Core Relational Tables (PostgreSQL)

Table	Key Columns	Partitioning
party	party_id, name, type, risk_tier, kyc_status, jurisdiction	None
account	account_id, party_id, account_type, opened_date, status	None
account_party_link	account_id, party_id, role, start_date, end_date	None
transaction	tx_id, debit_account_id, credit_account_id, amount, currency, timestamp, channel, status	Range by month on timestamp
alert	alert_id, tx_id, rule_id, severity, score, status, created_at	None
case_entity	case_id, entity_id, entity_type, link_reason	None
audit_log	event_id, actor_id, action, entity_type, entity_id, before, after, timestamp	Range by month on timestamp

Graph Schema (Apache AGE aml_graph)

Vertex Label	Key Properties
Account	account_id, type, risk_score, jurisdiction, status
Customer	customer_id, name, risk_tier, kyc_status, pep_flag, sanctions_flag
Merchant	merchant_id, name, category, jurisdiction
Device	device_id, type, fingerprint
Wallet	wallet_id, address, protocol, custody_type
Edge Label	Key Properties
---	---
SENT	amount, currency, count, first_seen, last_seen, suspicious
RECEIVED	amount, currency, count, first_seen, last_seen
OWNS	since, ownership_type
CONTROLS	since, control_type
SHARES_DEVICE	first_seen, last_seen, shared_count
LINKED_TO_CASE	case_id, link_reason, linked_at

Frontend Requirements (Module 4)

The ETL module has a lightweight admin view inside the portal for operations staff:[1][27]

- Pipeline run history table: status, run start/end, record counts, quality score, errors
- Pipeline DAG viewer (read-only iframe or embedded Airflow UI link)
- On-demand reprocessing trigger form
- Data quality metric charts (quality score trend, reject count by source)
- Alert configuration for quality threshold breaches

Backend Requirements (Module 4)

Endpoint	Method	Description
/etl/runs	GET	List recent pipeline runs with status and quality metrics
/etl/runs/{id}	GET	Detail for a specific pipeline run
/etl/reprocess	POST	Trigger reprocessing of a time window or source
/etl/quality	GET	Current and historical quality metrics
/etl/graph/status	GET	Graph projection lag, last sync, vertex/edge counts

Non-Functional Requirements

Performance

Requirement	Target	Rationale
Dashboard KPI load time	< 2 seconds for initial render	AML dashboards must support real-time risk monitoring without perceived lag[6][9]
API response time (p95) for read endpoints	< 500 ms	Consistent with sanctions screening API benchmarks[40]
Graph neighborhood query (2 hops, < 500 nodes)	< 1 second	Investigator productivity requires low-latency graph expansion[20]
Transaction ingestion throughput	≥ 2.4 million records/hour	Benchmark from scalable ETL pipeline research on 850M-record datasets[28]
Streaming graph update lag	< 5 seconds from transaction arrival to AGE projection	Consistent with streaming ingestion latency targets in ETL literature[28]
Real-time scoring budget	≤ 100 ms total per transaction, ML model ≤ 50 ms	Industry benchmark for payment authorization pipelines[41]
Cytoscape.js canvas render (1,000 elements)	< 3 seconds on average hardware	Cytoscape.js comfortably renders thousands of elements; complex styles and large viewports increase cost[22][23]

Requirement	Target	Rationale
Case queue load (TanStack Table, 500 rows)	< 1 second	Dense investigator UI must not slow analyst throughput

Scalability

- The system must scale horizontally to support a 10x increase in transaction volume without architectural changes to the core pipeline.[28][30]
- The FastAPI backend must support stateless deployment with multiple replicas behind a load balancer.
- The graph layer must support progressive subgraph expansion and server-side layout computation for graphs exceeding 1,000 elements to avoid browser-side performance degradation.[23][24]
- Apache Airflow DAGs must support concurrent execution with configurable parallelism per DAG and task pool.
- PostgreSQL must be deployed with read replicas for query-heavy analytics endpoints to isolate reporting load from transactional write paths.

Availability and Reliability

Requirement	Target
Core portal uptime	99.95% (< 4.4 hours downtime annually)
Transaction ingestion pipeline	99.9% (< 9 hours downtime annually)
Planned maintenance window	Maximum 2 hours per month, outside business hours
RTO (Recovery Time Objective)	< 1 hour for critical services
RPO (Recovery Point Objective)	< 15 minutes for transaction data

Consistent with financial application uptime guidance requiring 99.99% for payment gateways and payment authorization pipelines. [42][41]

Security

Requirement	Detail
Transport encryption	TLS 1.2+ enforced for all API, UI, and database traffic[43][44]
Authentication	OAuth2 / OpenID Connect with JWT; MFA required for all users; managed via Keycloak[45][44]
Authorization	RBAC enforced at API layer before any database or graph operation; least-privilege database roles per service[46][3]
Session management	Token expiry ≤ 1 hour; refresh token rotation; revocation on logout[44]
Data at rest	AES-256 encryption for database volumes and file attachments[42]
Input validation	All API inputs validated with Pydantic at FastAPI layer; parameterized Cypher queries only (no string interpolation)[47][44]
Secret management	Credentials injected via environment variables or secrets manager (Vault / AWS Secrets Manager); no hardcoded credentials[48][44]
Rate limiting	Applied at API gateway level; per-IP and per-user limits to prevent brute force and API abuse[44]
CORS	Strict allowlist limited to portal frontend origin[44]
Security headers	HSTS, X-Content-Type-Options, X-Frame-Options, CSP enforced via FastAPI middleware[44]
Dependency scanning	Automated SBOM and vulnerability scanning in CI/CD pipeline
Penetration testing	Annual third-party penetration test; quarterly internal security review

Compliance and Data Governance

- All audit log records must be immutable, append-only, and retained for a minimum of 7 years or as required by local regulation, whichever is longer.[11][3][15]
- The system must support GDPR data subject access requests (DSAR) and the right to erasure for non-SAR-filed customer records, with legal hold overrides.[3]
- Cross-border data transfer must comply with applicable data residency requirements; the system must support region-specific deployment configurations.[3]
- The system must generate evidence sufficient to satisfy regulatory inspection under FATF Recommendation 20, AMLD5/6 Article 33, and the EBA AML/CFT risk-based supervision guidelines.[11][3]
- Rule and model version metadata must be logged at the point of alert generation so historical dispositions remain explainable. [49][11]

Maintainability and Observability

- All services must emit structured JSON logs with correlation IDs spanning frontend request through backend service and database query.
- Distributed tracing (OpenTelemetry) must be instrumented on all FastAPI routes and ETL pipeline stages.
- Health check endpoints must be available on all services for load balancer and container orchestrator liveness and readiness probes.
- Deployments must be automated via CI/CD with automated test gates; critical bug fixes must be deployable within 12 hours of identification.[42]
- Graph query performance must be benchmarkable per endpoint, and slow-query logs (>500 ms) must be captured and alerted.
- Cypher queries must be version-controlled and reviewed as part of the codebase rather than constructed at runtime.[47]

Usability

- The portal must support all modern browsers (Chrome, Edge, Firefox, Safari) at current and one prior major version.
- The interface must follow WCAG 2.1 Level AA accessibility standards.
- Analyst workflows (triage, investigate, note, close) must be completable without mouse-only interaction; full keyboard navigation must be supported.
- The case management and dashboard modules must support responsive layout at tablet widths (768px+); graph investigation is desktop-only (1024px+) given the complexity of the canvas.
- Investigators must be able to complete alert triage and disposition in under 3 minutes for straightforward alerts; the UX must support batch disposition for alert queues exceeding 100 items.

Architecture Alignment Summary

Module	Frontend Stack	Backend Stack	Database
KPI Dashboard	Next.js, TanStack Query, ECharts	FastAPI, Temporal (alert triggers), Redis (cache)	PostgreSQL (aggregated metrics), Redis
Case Management	Next.js, TanStack Table, React Hook Form + Zod	FastAPI, Temporal (workflow), Keycloak (RBAC)	PostgreSQL (cases, audit), OpenSearch (full-text)
Graph Visualization	Cytoscape.js, React, TanStack Query	FastAPI (graph routes), AGE query service	PostgreSQL + Apache AGE (aml_graph)
ETL Pipeline	Airflow admin view (portal iframe)	Apache Airflow DAGs, Python ETL services	PostgreSQL + Apache AGE (write targets)

All modules share a common authentication boundary via Keycloak and a unified audit log service, ensuring that every user action across the portal is traceable, attributable, and regulatorily defensible.[3][14][15]

References

1. [An Analytics Solution for AML Transaction Monitoring | Ibrahim Maïga](#) - The required setup for this project included a PostgreSQL database configured locally as a staging g...
2. [AML Case Management: Process, Tools & Best Practices - Ondato](#) - Learn what AML case management is, how it works, key compliance steps, and best practices to manage ...
3. [How to set up case management for AML alerts process and roles](#) - Building an AML workflow with role matrix, RBAC, audit trail and chain of custody. Integration of TM...
4. [How to Implement Transaction Monitoring: Best Practices for AML ...](#) - Learn how to implement transaction monitoring to meet AML compliance standards. Discover best practi...
5. [4 Anti Money Laundering \(AML\) Requirements for Payment ...](#) - This guide explores four core AML requirements for payment processors, focusing on how these firms c...
6. [Key AML KPIs for RIAs: Preparing for FinCEN's 2028 Compliance Rule](#) - Below are eight critical AML KPIs for RIAs to define and monitor. These metrics cover the core compo...
7. [Measuring AML Software Effectiveness: Key Performance Indicators ...](#) - Regular KPI analysis helps identify areas where the AML system may be underperforming, allowing for ...
8. [Designing Data-Driven Dashboards to Enhance AML/CFT ... - LinkedIn](#) - This dashboard provides a comprehensive overview of general alert and UTR volumes while tracking int...
9. [AML Compliance Dashboard: What It Is And How To Build One](#) - Learn how to build an aml compliance dashboard to track AUSTRAC KPIs, automate data collection, and ...
10. ["How long does it take to pull your AML metrics?" | Jennifer Arnold](#) - ... response: "This is the information I have to give my board every month. It takes weeks to put to...
11. [Documentation Requirements in AML Compliance - AML Watcher](#) - Learn everything about AML documentation

- requirements; what to record, the retention rules, regulato...
12. [How the Right AML Case Management Software Changes Everything](#) - AML case management software is a specialised tool designed for detecting and managing potential mon...
 13. [AML Case Management \[Detailed Guide\] - iDenfy](#) - What are the Components of AML Case Management? · Transaction Monitoring · Alert Management · Case C...
 14. [AI Workflows for KYC AML Compliance | Jinba Blog](#) - This guide covers the complete automation stack for KYC, AML, and compliance workflows — from initia...
 15. [Compliance Audit Trail: What It Is and Why It Matters - Regly](#) - Learn what a compliance audit trail is, why it matters for fintechs, and how to build records that m...
 16. [SLA Metrics: How to Measure & Monitor SLA Performance](#) - First Response Time measures how quickly an agent acknowledges a new service request or incident aft...
 17. [\[PDF\] Scalable Graph Learning for Anti-Money Laundering: A First Look](#) - We review AML methods, current and emergent. We provide a first look at scalable graph convolutional...
 18. [Money Laundering Detection with AML Graph Analytics' Structuring ...](#) - Graph analytics does not require new datasets. It requires banks to connect the datasets they alread...
 19. [Application of graph databases and network analysis in AML](#) - Network analysis can easily identify suspicious patterns that are indicative of money laundering, su...
 20. [Network Analysis in AML Investigations - Flagright](#) - Explore the importance and methodologies of network analysis in AML investigations. Learn how to lev...
 21. [Graph analytics and anti-money laundering: 8 use cases - Linkurious](#) - In this article, we explore practical AML use cases where graph technology can enhance compliance ef...
 22. [Cytoscape.js: a graph theory library for visualisation and ...](#) - Abstract. Summary: Cytoscape.js is an open-source JavaScript-based graph library. Its most common us...
 23. [Performance Optimization | cytoscape/cytoscape.js | DeepWiki](#) - This document outlines strategies and techniques for improving the performance of Cytoscape.js appli...
 24. [How large data does cytoscape.js support to render? · Issue #875 · cytoscape/cytoscape.js](#) - I have 10729 nodes and 4185

- edges,when I render it,the view is blank. I can see the code it generate...
25. [Apache AGE: Graph Database Magic in PostgreSQL : r/bigdata](#) - Apache AGE: Graph Database Magic in PostgreSQL · Integrates graph and SQL for deep data insights. · ...
 26. [Apache AGE Graph Database](#) - Cypher queries are invoked by the "cypher()" function which returns a set of rows. The column values...
 27. [AML Data Integration | Multi-Format Connectors & ETL Datalake](#) - EyesClear's data integration layer connects to your core banking, payment, and messaging systems — c...
 28. [\[PDF\] designing scalable etl pipelines for multi-source graph database ...](#)
 29. [Examples Of Etl Pipelines...](#) - Explore diverse perspectives on ETL Pipeline with structured content covering tools, strategies, cha...
 30. [Designing Scalable ETL Pipelines for Multi-Source Graph Database ...](#) - Abstract. As organizations seek to harness complex, interconnected datasets from diverse sources, gr...
 31. [Understand the AML data model and requirements](#) - The AML data model schema is arranged into three areas: core banking data, risk investigation data, ...
 32. [AML input data model | Anti Money Laundering AI](#)
 33. [Building an Analytics Solution for Effective AML Transaction ...](#) - Explore a comprehensive ETL pipeline project designed to enhance AML transaction monitoring for fina...
 34. [How to ETL from One PostgreSQL to Another - Integrate.io](#) - ETL technologies are advancing at an astounding rate. And many tools are allowing you to consolidate...
 35. [Apache AGE Extension - PostgreSQL](#) - Vue d'ensemble d'Apache AGE et de ses fonctionnalités dans Azure Database pour PostgreSQL.
 36. [Harnessing the Power of Apache AGE: Integrating Graph Databases ...](#) - This article explores the integration of Apache AGE, a PostgreSQL extension for graph database funct...
 37. [Prefect vs Airflow vs ZenML: Best Platform to Run ML Pipelines](#) - In this Prefect vs Airflow vs ZenML article, we explain the difference between the three platforms a...
 38. [Apache Airflow vs Prefect: Which Scheduler for Analytics Engineering?](#)

39. [Prefect vs Airflow: A Comparison](#) - Explore Airflow vs Prefect, two data orchestration tools, and how they may be utilized to improve da...
40. [Sanctions Screening SLA: Response Time Requirements - Lenzo](#) - API-based screening platforms typically commit to 200–500ms response times per single-entity lookup ...
41. [Transaction Monitoring System: Architecture Guide - Redis](#) - Learn how transaction monitoring systems work, from real-time ingestion to ML-driven detection. Expl...
42. [What are non-functional requirements? Detailed guide + examples](#) - Non-functional requirements (NFRs) define the qualities of a system. In other words, they're not abo...
43. [Securing Your Graph Data: Authentication and Security in Apache Age](#) - Use trusted SSL certificates to prevent man-in-the-middle attacks. 3. Firewall Rules: Configure fire...
44. [A Practical Guide to FastAPI Security - David Muraya](#) - Secure your FastAPI app with a complete checklist. Covers XSS, SQL Injection, rate limiting, secure ...
45. [OAuth2 with Password \(and hashing\), Bearer with JWT ...](#) - FastAPI framework, high performance, easy to learn, fast to code, ready for production
46. [Securing Your Graph Database: Best Practices for Apache AGE](#) - It is crucial to keep Apache AGE and the supporting infrastructure up to date. Security patches that...
47. [The AGE Cypher Query Format — Apache AGE master documentation](#) - The AGE Cypher Query Format . Cypher queries are constructed using a function called cypher in ag_ca...
48. [Integrating Apache AGE with Python | Blog](#) - Writing the Python Code. Here is a sample Python script to connect to the PostgreSQL database, creat...
49. [\[PDF\] ACAMS White Paper AML Model Validation in Compliance with ...](#) - AML models also import data from other models or open-source databases such as watch lists. Data tra...